

StatPoker (Protótipo) – Documentação

José Pedro Silvério Assis



Naomi Marra Marcondes Ribeiro

O presente documento aborda o processo de desenvolvimento do protótipo do site de gerenciamento de torneios de pôquer "StatPoker", concebido por Victor Santos. O objetivo deste arquivo é detalhar as ferramentas empregadas no design e na programação do protótipo, bem como a estrutura do banco de dados, sua implementação e as classes, funções e objetos correspondentes no código.

O conceito inicial do site consistia em uma interface moderna e simplificada, sem funcionalidade de back-end. O objetivo era implementar operações CRUD (criação, leitura, atualização e exclusão) para torneios de pôquer e seus respectivos itens de caixa. Além disso, realizamos ajustes pontuais na interface original para conferir maior profissionalismo ao protótipo.

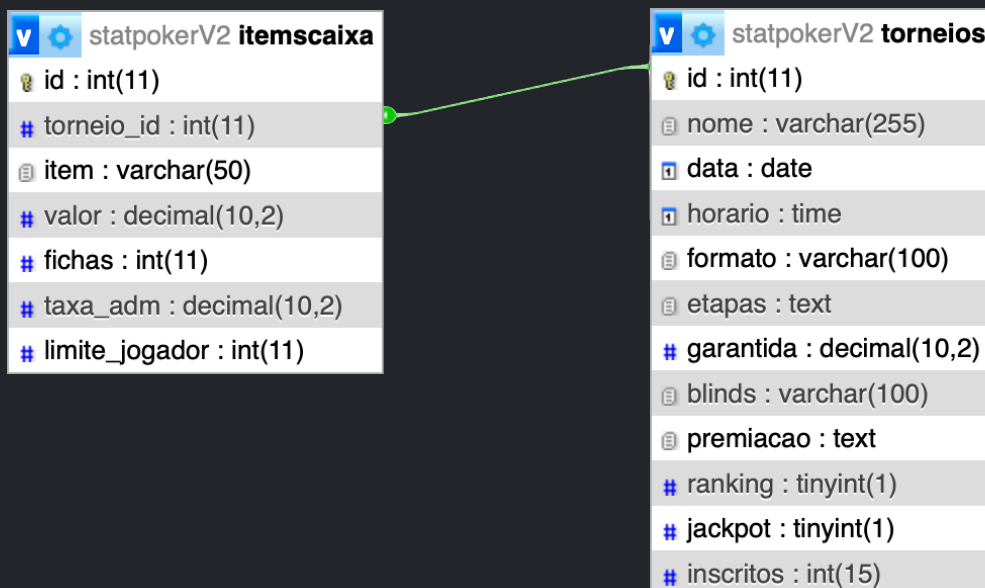
A versão beta atual foi elaborada em menos de uma semana e permanece em desenvolvimento; todo o conteúdo está sujeito a aprovação e poderá sofrer revisões futuras.

Banco de Dados

O banco de dados atual foi implementado no phpMyAdmin, integrado ao XAMPP, exclusivamente para fins de desenvolvimento. Sua manipulação é realizada via MySQL e compreende duas tabelas: "torneios", com 12 colunas, e "itenscaixa", com 7 colunas. A estrutura do banco é apresentada a seguir:

Relação das Tabelas

A tabela torneios tem uma relação de um para Muitos (ou 1:N / *One-to-Many*) com itens caixa.



Torneios (“torneios”)

Coluna	Tipo	Propósito
“id”	int(11)	Chave primária, auto-incrementada na criação de torneios. Utilizada para editar e deletar torneios, além de ser chave estrangeira na tabela “itenscaixa”.
“nome”	varchar(255)	Recebe o nome do torneio.
“data”	date	Recebe a data do torneio. (Se vazia, é auto-preenchida com a data atual).
“horario”	time	Recebe a hora do torneio. (Se vazia, é auto-preenchida com a hora atual).
“formato”	varchar(100)	Recebe o formato do torneio (“um dia” – padrão ou “classificatória com dia final”).
“etapas”	text	Recebe as etapas do torneio se o formato for “classificatória com dia final” (2, 3 ou 4 etapas + dia final).
“garantida”	decimal(10,2)	Recebe o valor garantido do torneio, limitado pelo HTML à incrementos de 500.
“blinds”	varchar(100)	Recebe a estrutura das blinds (“Padrão 20 min – late reg nível 8”, “Padrão 12 min – late reg nível 10”, “Padrão 30 min – late reg nível 6”).
“premiacao”	text	Recebe o esquema de pontuação (“Automática – 12% do field premiado”, “Percentuais fixos (50/30/20)” ou “Personalizada...”).
“ranking”	tinyint(1)	Booleano – True se “Pontua no ranking da temporada” (Padrão: True).
“jackpot”	tinyint(1)	Booleano – True se “Contribui para o jackpot” (Padrão: False) .
“inscritos”	int(15)	Recebe o número de inscritos, atualmente não é incrementado, apenas exibido.

Itens Caixa (“itenscaixa”)

Coluna	Tipo	Propósito
“id”	int(11)	Chave primária, auto-incrementada na criação de itens caixa. Utilizada para editar e deletar itens caixa.
“torneio_id”	int(11)	Chave estrangeira, recebe o id do torneio a qual o item corresponde.
“item”	varchar(50)	Recebe o nome do item (Padrões: “Buy-in” – obrigatório, “Re-entry”, “Add-on”).
“valor”	decimal(10,2)	Recebe o valor do item, o valor de “Buy-in” é exibido na lista de torneios.
“fichas”	int(11)	Recebe a quantidade de fichas.
“taxa_adm”	decimal(10,2)	Recebe a taxa.
“limite_jogador”	int(11)	Recebe o valor limite por jogador.

StatPoker (Protótipo) – Documentação

José Pedro Silvério Assis



Naomi Marra Marcondes Ribeiro

Estrutura de Arquivos

```
StatPoker/
├── index.php      <-- Landing page
├── src/
│   ├── Config/
│   │   └── Database.php    <-- Conexão com banco de dados
│   ├── Controller/
│   │   ├── excluir.php    <-- Intermediário entre o banco e o DAO
│   │   └── salvar.php      <-- Intermediário entre o banco e o DAO
│   ├── DAO/
│   │   ├── ItemDAO.php    <-- Objeto de acesso de dados para itens Caixa
│   │   └── TorneioDAO.php  <-- Objeto de acesso de dados para torneios
│   ├── Model/
│   │   ├── ItemCaixa.php  <-- Classe Itens Caixa com getters e setters
│   │   └── Torneio.php    <-- Classe Torneios getters e setters
│   └── Views/
│       ├── home/
│       │   └── home.php    <-- Placeholder de dashboard com dados gerais
│       ├── layouts/
│       │   ├── footer.php <-- Placeholder de footer
│       │   ├── header.php <-- Header e barra lateral reutilizáveis
│       └── torneios/
│           ├── criar.php   <-- Formulário para criar novos torneios
│           └── lista.php   <-- Lista de todos os torneios
├── .
└── public/
    ├── assets/
    │   ├── fonts/        <-- Fontes customizadas
    │   ├── icons/        <-- Favicon
    │   └── imgs/         <-- Logo e outras imagens
    ├── css/
    │   ├── base/
    │   │   └── chip-animation.css <-- Animação de fundo
    │   ├── base/
    │   │   ├── fonts.css      <-- Importa fontes
    │   │   ├── normalize.css  <-- Remove estilos padrão de navegador
    │   │   ├── page.css       <-- Edita o layout básico da pagina
    │   │   └── variables.css   <-- Guarda variáveis de cor
    │   ├── blocks/
    │   │   ├── form.css       <-- Edita o formulário
    │   │   ├── header.css     <-- Edita o header
    │   │   ├── landing.css    <-- Edita o landing
    │   │   ├── list.css       <-- Edita a lista de torneios
    │   │   └── sidebar.css     <-- Edita a barra lateral
    │   └── main.css           <-- Importa os outros arquivos .css
    └── js/
        ├── torneio-filtro.js <-- Script para filtrar torneios na lista
        └── torneio-novo.js   <-- Script para atualizar a lista de blinds
```

Classe Database.php

Módulo de conexão PDO e padrão Singleton.

A classe Database é responsável por centralizar e gerenciar a conexão com o banco de dados MySQL. Ela impede a criação de múltiplas instância desnecessárias da mesma conexão durante uma única requisição, otimizando o consumo de memória e conexões no servidor.

```
2. /**
3.  * Classe responsável pelo gerenciamento da conexão com o banco
4.  * Ela usa atributos static para que a variável instance não seja reiniciada ou
5.  * apagada enquanto script php está em execução.
6.  * Implementa o padrão Singleton para garantir que apenas uma
7.  * instância da conexão PDO seja criada durante a execução da aplicação
8.  */
9. class Database {
10.     //Endereço do servidor do banco de dados
11.     private static string $host = "localhost";
12.
13.     //Usuario para autentificação no banco
14.     private static string $user = "root";
15.
16.     //Senha para autentificação no banco
17.     private static string $pass = "";
18.
19.     //Nome do banco de dados
20.     private static string $db = "StatPoker";
21.
22.     //Conjunto de caracteres da conexão
23.     private static string $charset = 'utf8mb4';
24.
25.     //Guarda unica instancia do objeto PDO
26.     private static ?PDO $instance = null;
27.
28.     /**
29.      * Retorna a instancia unica da conexão PDO
30.      *
31.      * Caso a conexão ainda não exista, ela será inicializado
32.      * com as configurações definidas na classe
33.      *
34.      * Return self::$instance - Retorna objeto de conexão PDO ativo
35.      *
36.      * Throws PDOException - Se a conexão com banco falhar
37.      */
38.     public static function getConexao() : PDO {
39.         if(self::$instance===null) {
40.
41.             //Data source name, string que contem informações necessárias
42.             //para conectar php a um banco usando extensão pdo
43.             $dsn = "mysql:host=" . self::$host . ";dbname=" . self::$db .
";charset=" . self::$charset . ";";
44.
45.             //PDO config options
46.             $options = [
47.                 //Lança exceções em erros de SQL
48.                 PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
49.
50.                 //Retorna dados como arrays associativos
51.                 PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
52.
53.                 //Desativa emulação para uso de prepared statements reais
54.                 PDO::ATTR_EMULATE_PREPARES => false
55.             ];
56.
57.             try {
58.                 //Inicializa conexão com banco
59.                 self::$instance = new PDO($dsn, self::$user, self::$pass,
$options);
60.             } catch (PDOException $e) {
61.                 throw new PDOException("Conexão com banco de dados falhou: " . $e-
>getMessage(), (int)$e->getCode());
62.             }
63.
64.             return self::$instance;
65.         }
66.     }
```

Sistema CRUD

(Criação, Leitura, Atualização e Deleção de Usuário).

O sistema de gestão de torneios do StatPoker segue uma arquitetura em camadas (MVC simplificado), separando a apresentação, a lógica de negócio e o acesso a dados. A camada de Model é composta pelas classes Torneio e ItemCaixa, que representam as entidades do domínio e encapsulam suas regras (como o cálculo dinâmico de status do torneio ou a garantia de valores não-negativos).

A camada de DAO (TorneioDAO e ItemDAO) concentra todas as operações de banco (inserir, atualizar, buscar e excluir), utilizando prepared statements via PDO — cuja conexão é gerenciada de forma centralizada e única (Singleton) pela classe Database.

A camada de View/Controller é dividida por operação: lista.php exibe todos os torneios cadastrados (Read), consultando também os itens de caixa de cada um para montar informações como buy-in e status; criar.php serve tanto para exibir o formulário de criação quanto para pré-carregar os dados de um torneio existente quando um id é passado por GET (preparando o Update); salvar.php recebe os dados do formulário via POST e decide, com base na presença do id, se deve inserir um novo torneio ou atualizar um existente (Create/Update), tratando também os itens de caixa associados; e excluir.php recebe um id via GET ou POST, valida-o e aciona a remoção do registro (Delete), redirecionando sempre para a listagem ao final. Essa separação permite que a lógica de persistência fique isolada nos DAOs, enquanto os controllers cuidam apenas do fluxo de requisição/resposta e as views da apresentação dos dados.

Controller salvar.php

Intermedia criação entre o usuário e o sistema.

Grava torneios e itens de caixa via POST (redirecionando em outros métodos). Ele cria um objeto Torneio com os dados recebidos (nome, data, horário, formato, valor garantido, blinds, premiação, ranking e jackpot) e usa TorneioDAO::salvar() para inserir ou atualizar conforme o ID. Depois, decodifica o JSON de itens_caixa, normaliza os valores (fichas, taxa e limite) evitando erros no MySQL, cria objetos ItemCaixa e os salva com ItemDAO::salvar(), vinculados ao torneio. Por fim, redireciona para a lista.

```
2. /**
3.  * Controller responsavel pela Inserção/Atualização de torneios
4.  * Recebe ID do torneio, valida e executa modificações via TorneioDAO
5.  */
6. //Apontamento até a pasta raiz
7. if (!defined('BASE_URL')) define('BASE_URL', '/statpoker/S');
8.
9. //Importação de dependências necessarias
10.require_once __DIR__ . '/../Config/Database.php';
11.require_once __DIR__ . '/../Model/Torneio.php';
12.require_once __DIR__ . '/../Model/ItemCaixa.php';
13.require_once __DIR__ . '/../DAO/TorneioDAO.php';
14.require_once __DIR__ . '/../DAO/ItemDAO.php';
15.
16.//Inicializa a conexão com o banco e a camada DAO
17.$pdo = Database::getConexao();
18.$tDAO = new TorneioDAO($pdo);
19.$iDAO = new ItemDAO($pdo);
20.
21.$t = new Torneio();
22.
23.//Garante que codigo só rode se dados tiverem sido enviados por form via post
24.if ($ _SERVER['REQUEST METHOD'] !== 'POST') {
25.    header('Location:-' . BASE_URL . '/src/Views/torneios/lista.php');
26.    exit;
27.}
28.
29.//Verificação de id, verifica se campo $_POST['id'] existe e não esta vazio
30.if (!empty($ _POST['id'])) {
31.    //Converte string id em int e grava valor dentro do objeto
32.    $t->setId((int)$ _POST['id']);
33.}
34.
```

```

35.
36.     if (is_array($itensArray)) {
37.         //Limpa itens caixa para evitar duplicação
38.         $iDAO->deletarPorTorneio($torneioId);
39.
40.         foreach ($itensArray as $dado) {
41.
42.             $item = new ItemCaixa();
43.             $item->setTorneioId($torneioId);
44.             $item->setItem($dado['item'] ?? '');
45.             $item->setValor((float)($dado['valor'] ?? 0));
46.
47.             //Garantia: se for vazio/null, vira 0 (evita o erro NOT NULL no MySQL)
48.             $item->setFichas((int)($dado['fichas'] ?? 0));
49.
50.             $item->setTaxaAdm(isset($dado['taxa']) ? (float)$dado['taxa'] : 0.0);
51.             $item->setLimiteJogador(isset($dado['limite']) ? (int)$dado['limite'] :
0);
52.
53.             //Executa o salvamento dos itens de caixa no banco de dados
54.             $iDAO->salvar($item);
55.         }
56.     }
57. }
58.
59. //Redireciona de volta para a lista
60. header('Location: ' . BASE_URL . '/src/Views/torneios/lista.php');
61. exit;
62. ?>

```

Controller Excluir (PHP) Intermedia exclusão entre o usuário e o sistema.

Atua como controller responsável pela exclusão de um torneio. Ele recebe uma requisição com o ID do torneio, valida os dados, adiciona a exclusão no banco via TorneioDAO e redireciona o usuário de volta para listagem

```

2.  /*
3.  * Controller responsável pela exclusão de torneios
4.  * Recebe ID do torneio, valida e executa remoção via TorneioDAO
5.  */
6.  if (!defined('BASE_URL')) {
7.      define('BASE_URL', '/statpoker/S');
8.  }
9.
10. //Importação de dependências necessárias
11. require_once __DIR__ . '/../src/Config/Database.php';
12. require_once __DIR__ . '/../src/DAO/TorneioDAO.php';
13.
14. //Captura ID vindo de requisição GET ou POST
15. $id = $_GET['id'] ?? $_POST['id'] ?? null;
16. //Garante que ID exista e seja um número inteiro válido
17. if (!$id || !filter_var($id, FILTER_VALIDATE_INT)) {
18.     header('Location: ' . BASE_URL . '/src/Views/torneios/lista.php');
19.     exit;
20. }
21.
22. //Inicializa a conexão com o banco e a camada DAO
23. $pdo = Database::getConnection();
24. $dao = new TorneioDAO($pdo);
25.
26. //Executa a exclusão no banco de dados
27. $dao->excluir((int)$id);
28.
29. //Redirecionamento para página inicial de torneios
30. header('Location: ' . BASE_URL . '/src/Views/torneios/lista.php');
31. exit;

```